LAPORAN PRAKTIKUM
POSTTEST 4
ALGORITMA PEMROGAMAN LANJUT

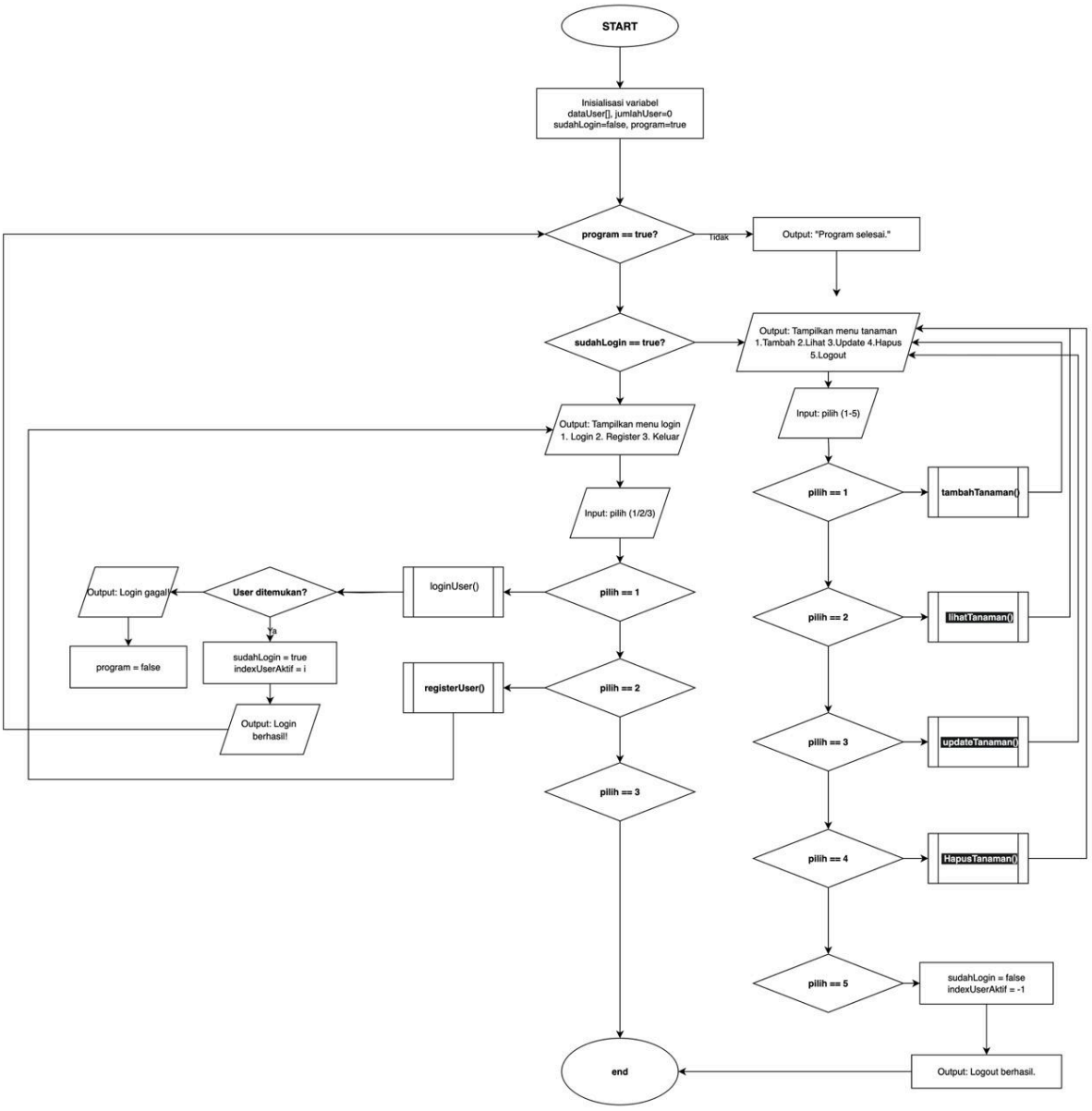
Disusun oleh:

Kikania Agarista (2509106086)

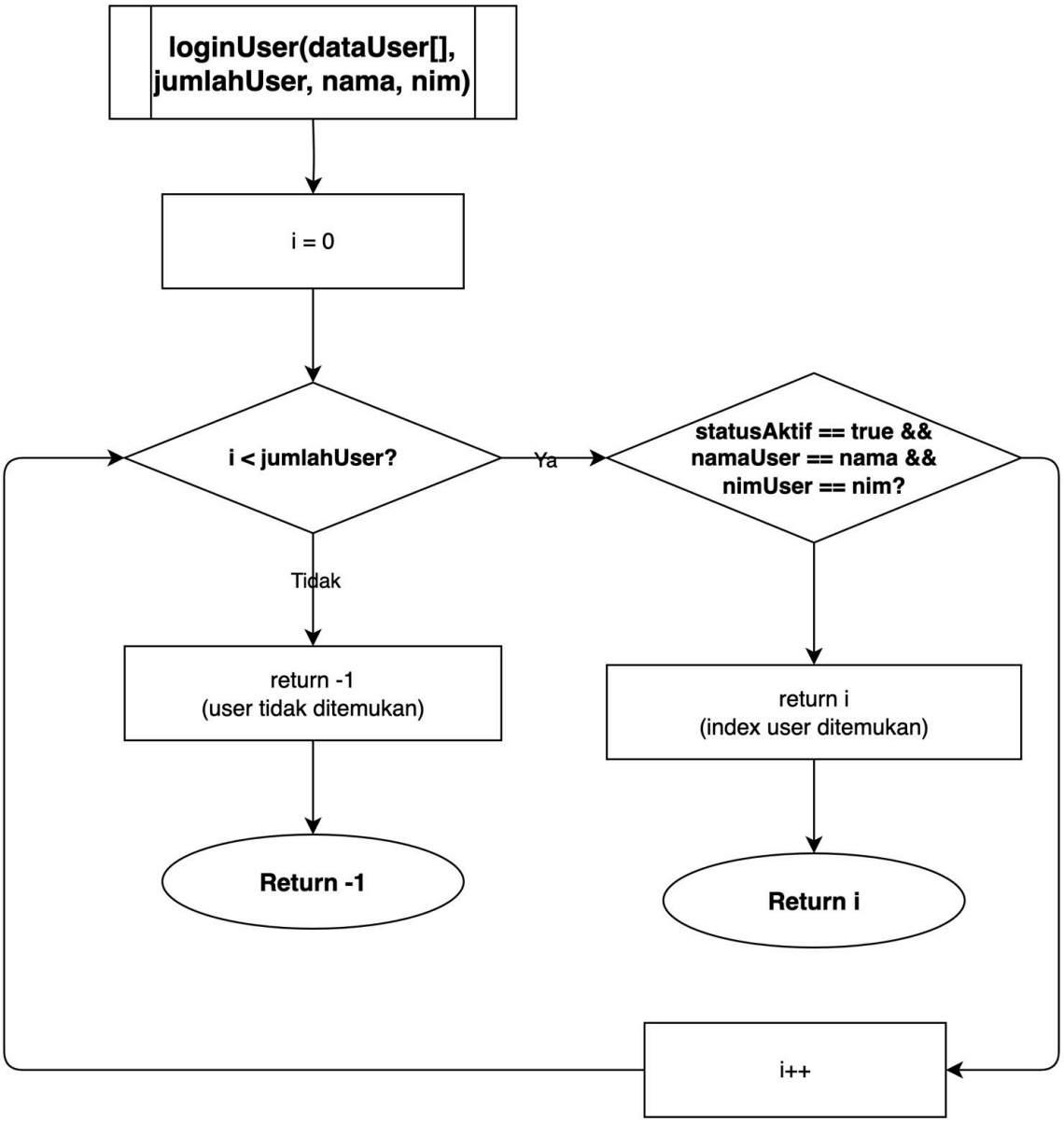
Kelas (B2'25)

PROGRAM STUDI INFORMATIKA
UNIVERSITAS MULAWARMAN
2026

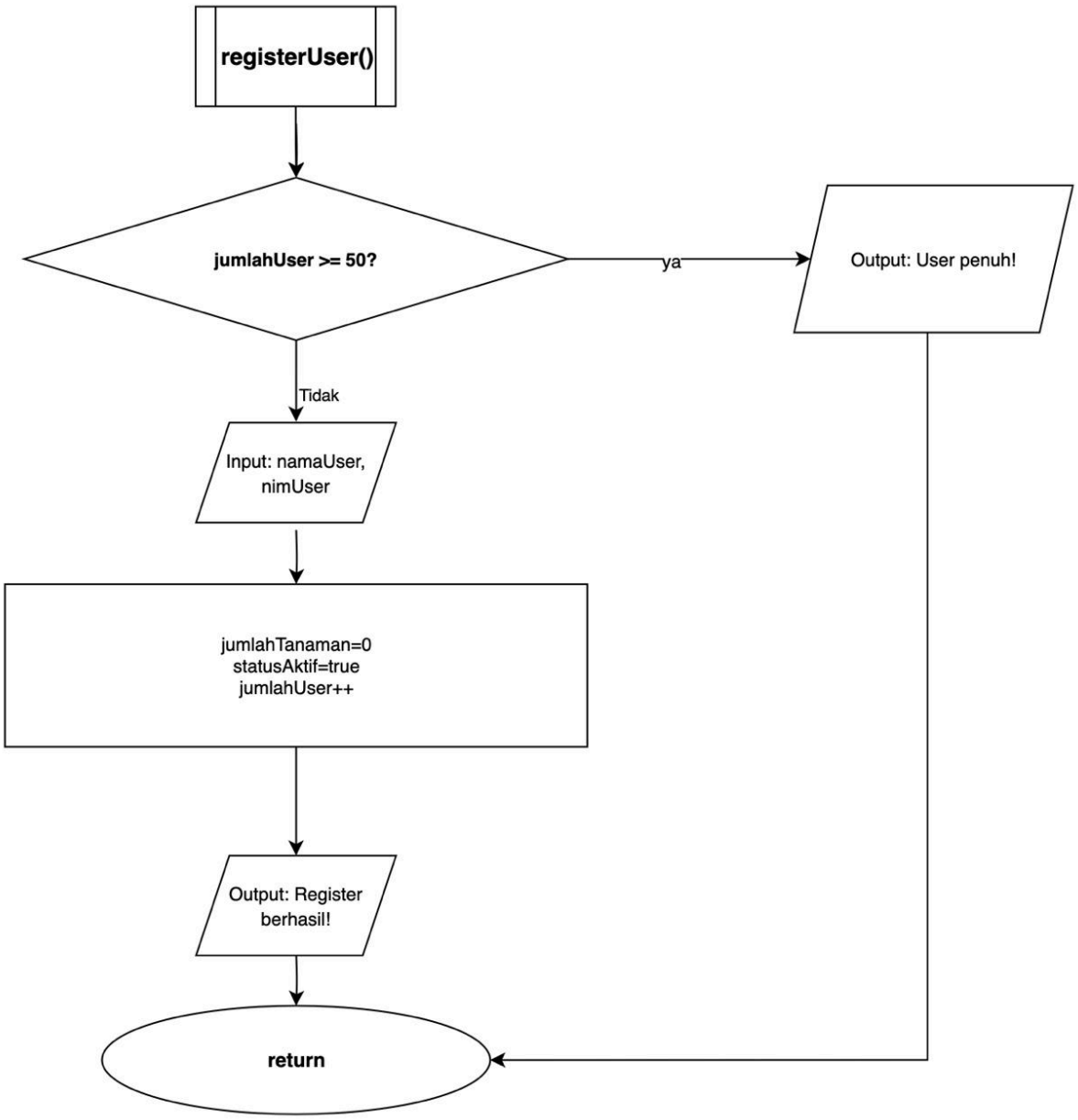
A. Flowchart



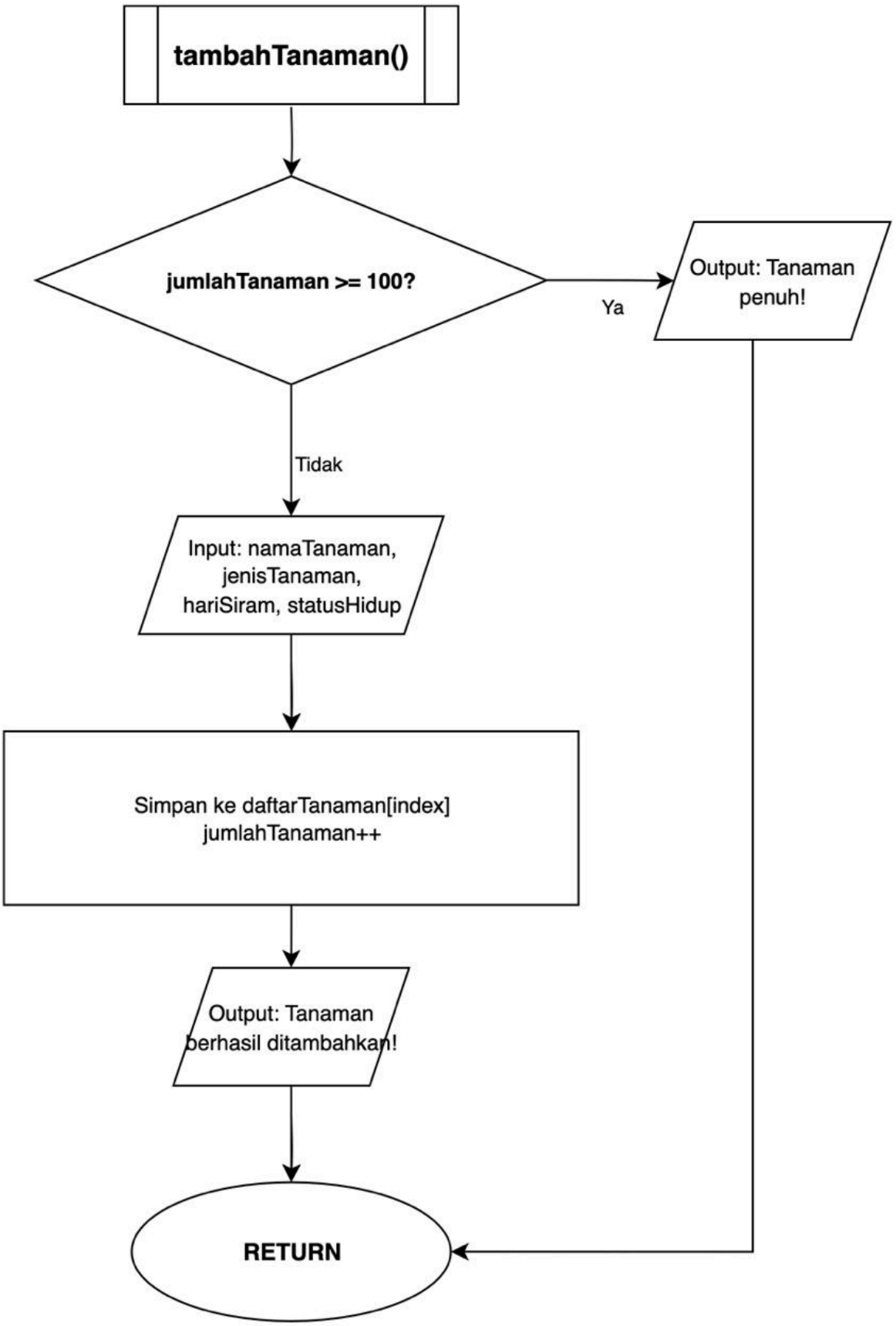
Gambar 1.1


loginUser(dataUser[], jumlahUser, nama, nim)
i = 0
i < jumlahUser?
Ya
statusAktif == true && namaUser == nama && nimUser == nim?
Tidak
return -1 (user tidak ditemukan)
return i (index user ditemukan)
Return -1
Return i
i++

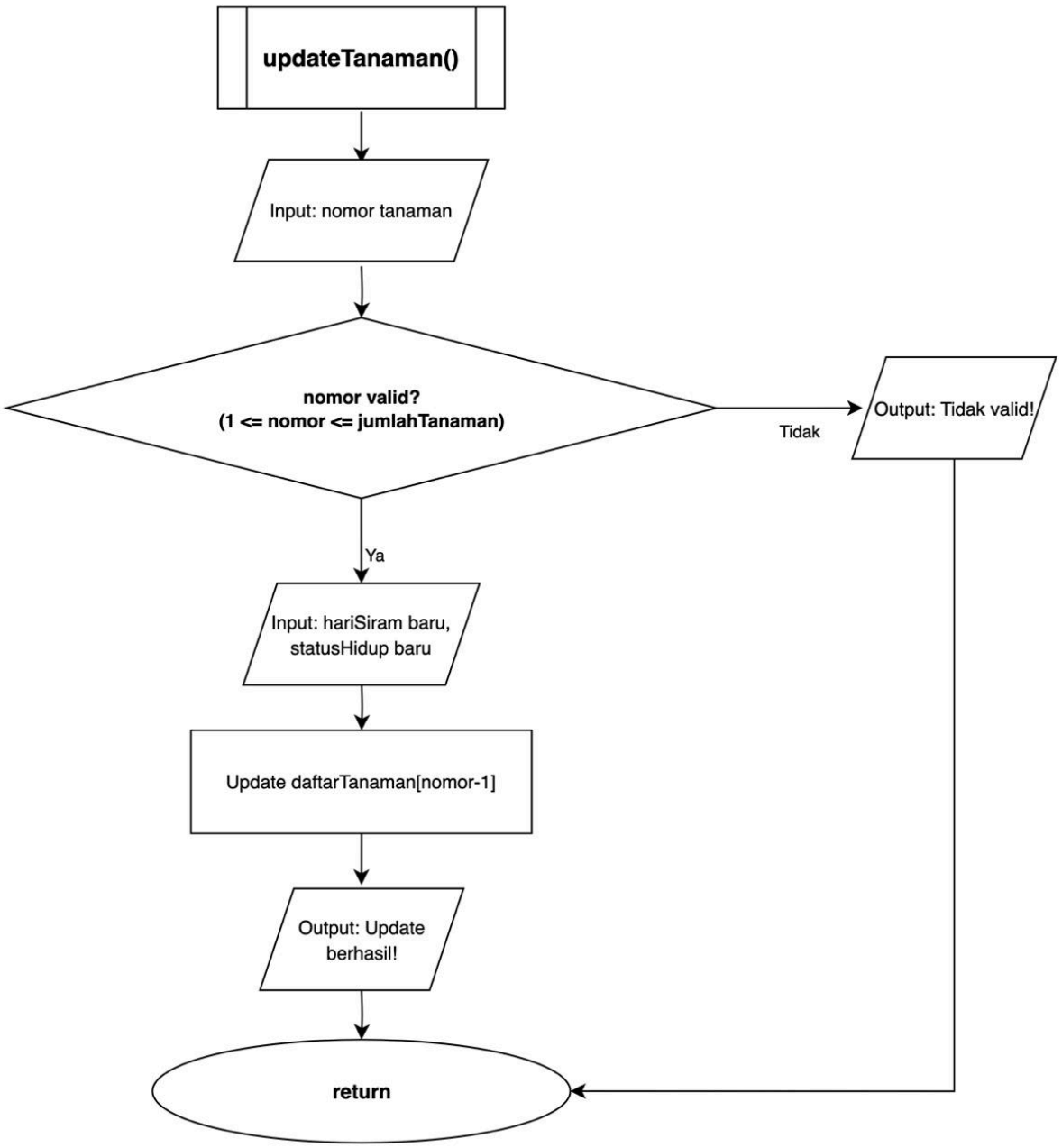
Gambar 1.2


registerUser()
jumlahUser >= 50?
ya
Output: User penuh!
Tidak
Input: namaUser, nimUser
jumlahTanaman=0
statusAktif=true
jumlahUser++
Output: Register berhasil!
return

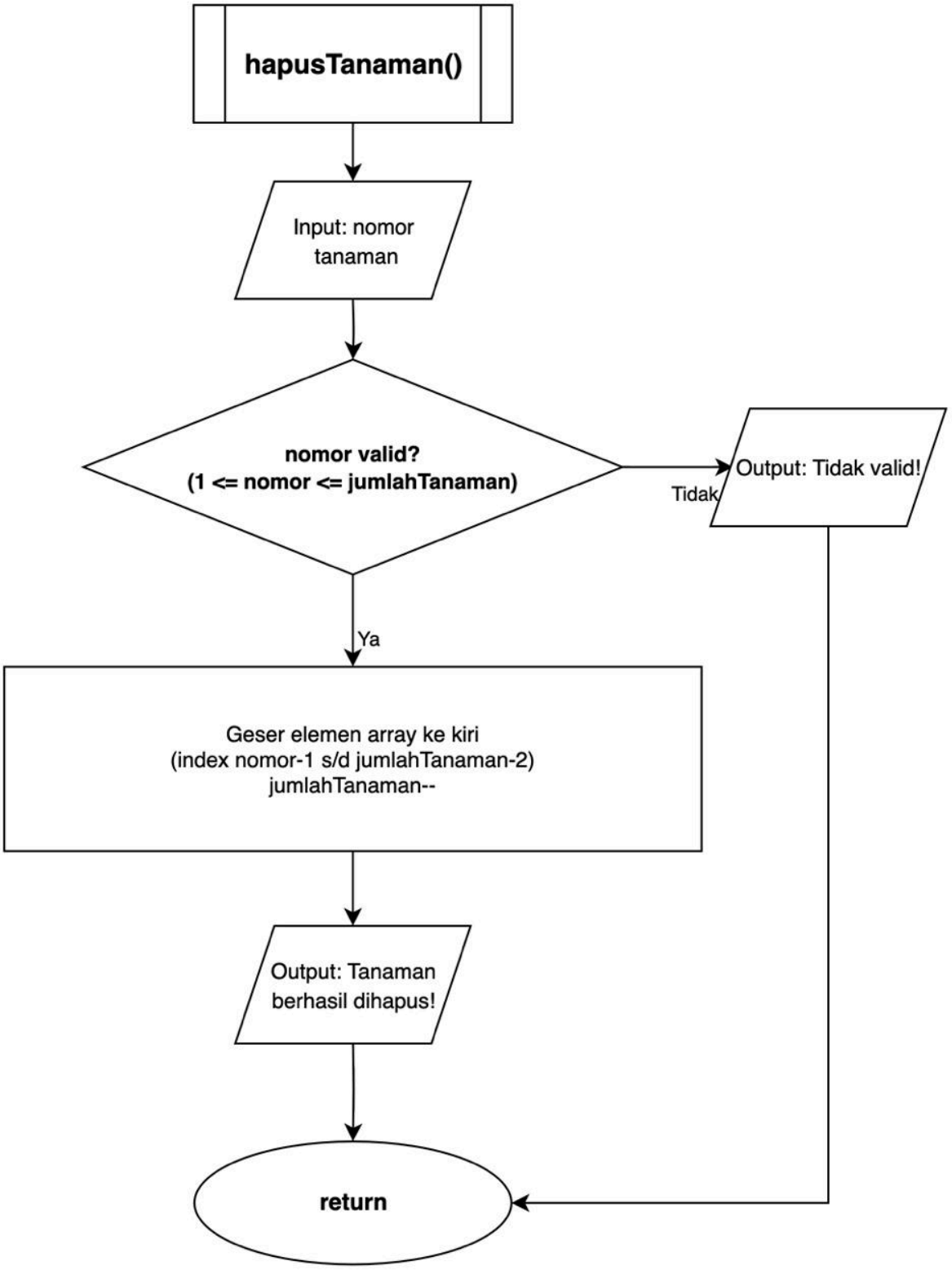
Gambar 1.3


tambahTanaman()
jumlahTanaman >= 100?
Ya
Output: Tanaman penuh!
Tidak
Input: namaTanaman, jenisTanaman, hariSiram, statusHidup
Simpan ke daftarTanaman[index]
jumlahTanaman++
Output: Tanaman berhasil ditambahkan!
RETURN

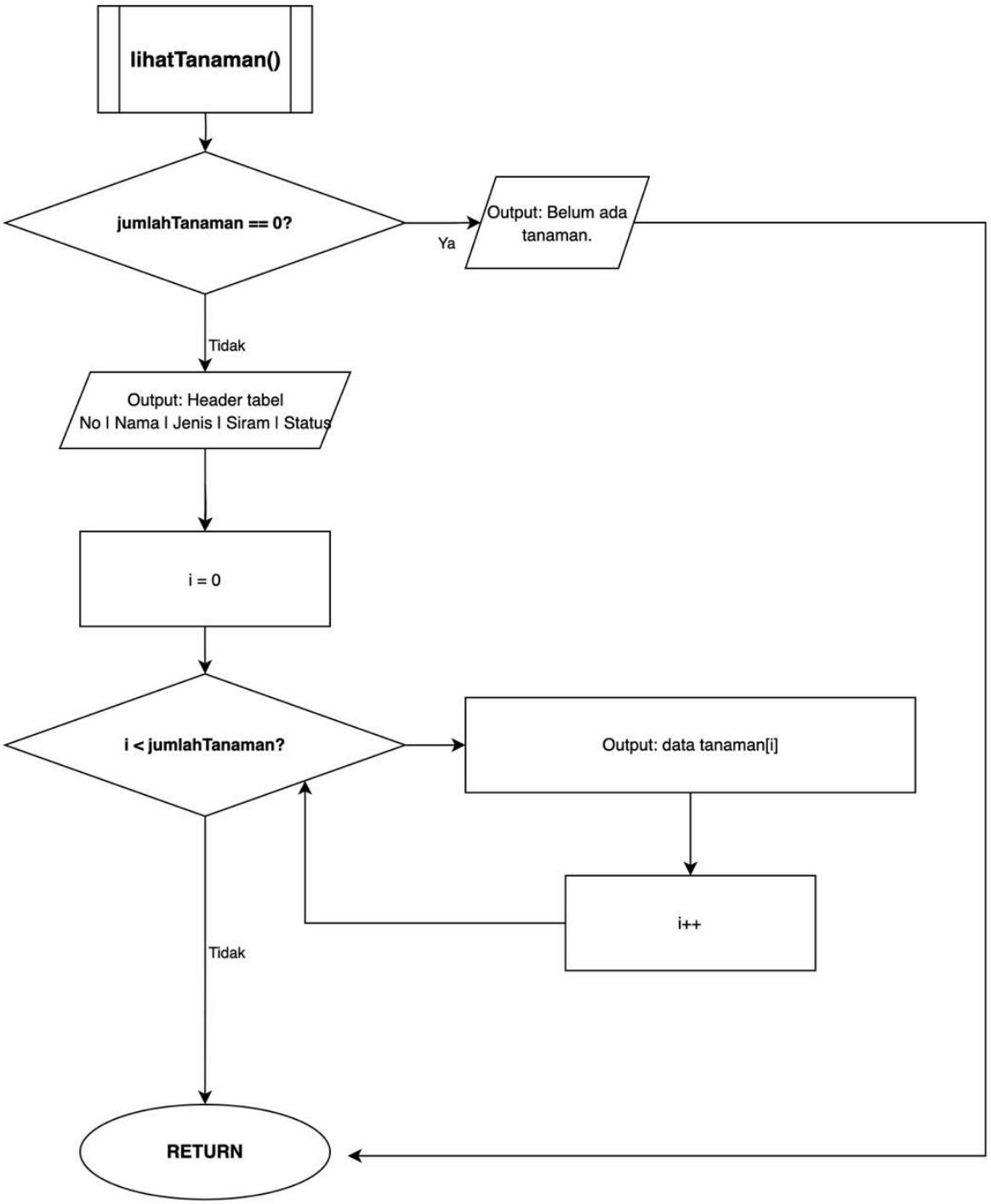
Gambar 1.4


updateTanaman()
Input: nomor tanaman
nomor valid?
(1 <= nomor <= jumlahTanaman)
Tidak
Output: Tidak valid!
Ya
Input: hariSiram baru,
statusHidup baru
Update daftarTanaman[nomor-1]
Output: Update
berhasil!
return

Gambar 1.5


hapusTanaman()
Input: nomor tanaman
nomor valid?
(1 <= nomor <= jumlahTanaman)
Tidak
Output: Tidak valid!
Ya
Geser elemen array ke kiri
(index nomor-1 s/d jumlahTanaman-2)
jumlahTanaman--
Output: Tanaman berhasil dihapus!
return

Gambar 1.6


lihatTanaman()
jumlahTanaman == 0?
Ya
Output: Belum ada tanaman.
Tidak
Output: Header tabel
No | Nama | Jenis | Siram | Status
i = 0
i < jumlahTanaman?
Output: data tanaman[i]
i++
Tidak
RETURN

Gambar 1.7

B. Penjelasan Singkat

Program Sistem Manajemen Tanaman Pribadi ini merupakan pengembangan dari Posttest 2 dengan menerapkan konsep Pointer sesuai Modul 4. Perubahan utamanya adalah penggunaan pointer pada struct User dan Tanaman, di mana array statis (Tanaman daftarTanaman[100]) diubah menjadi alokasi dinamis menggunakan new dan delete agar lebih efisien dalam penggunaan memori. Program ini juga menerapkan Address-of Operator (&) untuk mengirim alamat variabel asli ke fungsi, Dereference Operator (*) untuk mengakses nilai di alamat memori, serta Arrow Operator (->) untuk mengakses atribut struct melalui pointer. Dengan pendekatan modular programming, setiap fitur seperti Login, Register, dan CRUD dipisah menjadi fungsi-fungsi independen sehingga kode lebih rapi, mudah dipahami, dan sesuai dengan materi yang telah dipelajari di modul.

C. Source Code

```
#include <iostream>
#include <string>
using namespace std;

struct Tanaman {
    string namaTanaman;
    string jenisTanaman;
    int hariTerakhirSiram;
    bool statusHidup;
};

struct User {
    string namaUser;
    string nimUser;
    Tanaman* daftarTanaman;
    int jumlahTanaman;
    bool statusAktif;
};
```

1. Fungsi LoginUser

```
int loginUser(User* dataUser, int jumlahUser, string nama, string nim) {
    for (int i = 0; i < jumlahUser; i++) {
        if ((dataUser + i)->statusAktif &&
            (dataUser + i)->namaUser == nama &&
            (dataUser + i)->nimUser == nim) {
            return i;
        }
    }
```

```
    return -1;
}
```

2. Fungsi RegisterUser

```
void registerUser(User* dataUser, int &jumlahUser) {
    if (jumlahUser >= 50) {
        cout << "User penuh!" << endl;
        return;
    }
```

3. Pointer Aritmatika

```
    cin.ignore();
    cout << "Nama: ";
    getline(cin, (dataUser + jumlahUser)->namaUser);

    cout << "NIM: ";
    getline(cin, (dataUser + jumlahUser)->nimUser);
```

4. Inisialisasi Pointer dalam daftarTanaman

```
    (dataUser + jumlahUser)->daftarTanaman = new Tanaman[100];
    (dataUser + jumlahUser)->jumlahTanaman = 0;
    (dataUser + jumlahUser)->statusAktif = true;

    jumlahUser++;

    cout << "Register berhasil!" << endl;
    return;
}
```

5. Fungsi Tambah Tanaman

```
void tambahTanaman(User* user) {
    if (user->jumlahTanaman >= 100) {
        cout << "Tanaman penuh!" << endl;
        return;
    }

    int index = user->jumlahTanaman;

    cin.ignore();
    cout << "Nama Tanaman: ";
    getline(cin, (user->daftarTanaman + index)->namaTanaman);
```

```
    cout << "Jenis: ";
    getline(cin, (user->daftarTanaman + index)->jenisTanaman);

    cout << "Hari Siram (1-7): ";
    cin >> (user->daftarTanaman + index)->hariTerakhirSiram;

    cout << "Status Hidup (1=Ya, 0=Tidak): ";
    cin >> (user->daftarTanaman + index)->statusHidup;

    user->jumlahTanaman++;

    cout << "Tanaman berhasil ditambahkan!" << endl;
    return;
```

6. Fungsi Lihat Daftar Tanaman

```
void lihatTanaman(User* user) {
    if (user->jumlahTanaman == 0) {
        cout << "Belum ada tanaman." << endl;
        return;
    }

    cout << "No | Nama | Jenis | Siram | Status" << endl;
    cout << "----------------------------------------" << endl;

    for (int i = 0; i < user->jumlahTanaman; i++) {
        cout << i + 1 << " | "
             << (user->daftarTanaman + i)->namaTanaman << " | "
             << (user->daftarTanaman + i)->jenisTanaman << " | "
             << (user->daftarTanaman + i)->hariTerakhirSiram << "
| "
             << ((user->daftarTanaman + i)->statusHidup ? "Hidup"
: "Mati")
             << endl;
    }
    cout << "----------------------------------------" << endl;
    return;
}
```

7. Fungsi Update Tanaman

```
void updateTanaman(User* user) {
    int nomor;
    cout << "Nomor tanaman: ";
    cin >> nomor;
```

```
    if (nomor <= 0 || nomor > user->jumlahTanaman) {
        cout << "Tidak valid!" << endl;
        return;
    }

    int index = nomor - 1;

    cout << "Hari Siram Baru: ";
    cin >> (user->daftarTanaman + index)->hariTerakhirSiram;

    cout << "Status Hidup Baru (1=Ya, 0=Tidak): ";
    cin >> (user->daftarTanaman + index)->statusHidup;

    cout << "Update berhasil!" << endl;
    return;
}
```

8. Fungsi Hapus Tanaman

```
void hapusTanaman(User* user) {
    int nomor;
    cout << "Nomor tanaman: ";
    cin >> nomor;

    if (nomor <= 0 || nomor > user->jumlahTanaman) {
        cout << "Tidak valid!" << endl;
        return;
    }

    for (int i = nomor - 1; i < user->jumlahTanaman - 1; i++) {
        *(user->daftarTanaman + i) = *(user->daftarTanaman + i + 1);
    }

    user->jumlahTanaman--;

    cout << "Tanaman berhasil dihapus!" << endl;
    return;
}
```

9. Fungsi Destruktor (cleanUp Memori)

```
void cleanupMemory(User* dataUser, int jumlahUser) {
    for (int i = 0; i < jumlahUser; i++) {
        if ((dataUser + i)->daftarTanaman != nullptr) {
            delete[] (dataUser + i)->daftarTanaman;
        }
```

```
    }
    return;
}
```

10. Main Program

```
int main() {
    User* dataUser = new User[50];
    int jumlahUser = 0;
    int indexUserAktif = -1;

    bool sudahLogin = false;
    bool program = true;

    for (int i = 0; i < 50; i++) {
        (dataUser + i)->daftarTanaman = nullptr;
        (dataUser + i)->statusAktif = false;
    }

    while (program) {
        if (!sudahLogin) {
            cout << "\n=== LOGIN MENU ===" << endl;
            cout << "1. Login" << endl;
            cout << "2. Register" << endl;
            cout << "3. Keluar" << endl;
            cout << "Pilihan: ";

            int pilih;
            cin >> pilih;

            if (pilih == 1) {
                string nama, nim;

                cin.ignore();
                cout << "Nama: ";
                getline(cin, nama);

                cout << "NIM: ";
                getline(cin, nim);

                indexUserAktif = loginUser(dataUser, jumlahUser, nama, nim);

                if (indexUserAktif != -1) {
                    sudahLogin = true;
                    cout << "Login berhasil!" << endl;
                } else {
                    cout << "Login gagal!" << endl;
                }
```

```
        } else if (pilih == 2) {
            registerUser(dataUser, jumlahUser);

        } else if (pilih == 3) {
            program = false;
        }
    } else {

        cout << "\n=== MENU TANAMAN ===" << endl;
        cout << "1. Tambah" << endl;
        cout << "2. Lihat" << endl;
        cout << "3. Update" << endl;
        cout << "4. Hapus" << endl;
        cout << "5. Logout" << endl;
        cout << "Pilihan: ";

        int pilih;
        cin >> pilih;

        if (pilih == 1) {
            tambahTanaman(&dataUser[indexUserAktif]);
        } else if (pilih == 2) {
            lihatTanaman(&dataUser[indexUserAktif]);
        } else if (pilih == 3) {
            updateTanaman(&dataUser[indexUserAktif]);
        } else if (pilih == 4) {
            hapusTanaman(&dataUser[indexUserAktif]);
        } else if (pilih == 5) {
            sudahLogin = false;
            indexUserAktif = -1;
            cout << "Logout berhasil." << endl;
        }
    }
}
```

D. Output Program

```
=== LOGIN MENU ===
1. Login
2. Register
3. Keluar
```

Gambar 3.1 *Output Menu Login*

```
=== LOGIN MENU ===
1. Login
2. Register
3. Keluar
Pilihan: 2
Nama: Atma
NIM: 023
Register berhasil!
```

Gambar 3.2 *Output Menu Register*

```
=== LOGIN MENU ===
1. Login
2. Register
3. Keluar
Pilihan: 1
Nama: Atma
NIM: 023
Login berhasil!
```

Gambar 3.3 *Output Menu Login*

```
=== MENU TANAMAN ===
1. Tambah
2. Lihat
3. Update
4. Hapus
5. Logout
Pilihan:
```

Gambar 3.4 *Output Menu Tanaman*

```
=== MENU TANAMAN ===
1. Tambah
2. Lihat
3. Update
4. Hapus
5. Logout
Pilihan: 1
Nama Tanaman: anggrek
Jenis: bunga
Hari Siram (1-7): 3
Status Hidup (1=Ya, 0=Tidak): 1
Tanaman berhasil ditambahkan!
```

Gambar 3.5 *Output Menu Tambah Tanaman*

```
=== MENU TANAMAN ===
1. Tambah
2. Lihat
3. Update
4. Hapus
5. Logout
Pilihan: 2
No | Nama | Jenis | Siram | Status
---------------------------------------
1 | anggrek | bunga | 3 | Hidup
2 | kamboja | bunga | 5 | Hidup
3 | kol | sayur | 6 | Mati
4 | sawi | sayur | 2 | Mati
5 | apel | buah | 1 | Mati
6 | kacang tanah | kacang | 4 | Hidup
---------------------------------------
```

Gambar 3.6 *Output Menu Lihat Tanaman*

```
=== MENU TANAMAN ===
1. Tambah
2. Lihat
3. Update
4. Hapus
5. Logout
Pilihan: 3
Nomor tanaman: 1
Hari Siram Baru: 7
Status Hidup Baru (1=Ya, 0=Tidak): 0
Update berhasil!
```

Gambar 3.7 *Output Menu Update Tanaman*

```
=== MENU TANAMAN ===
1. Tambah
2. Lihat
3. Update
4. Hapus
5. Logout
Pilihan: 2
No | Nama | Jenis | Siram | Status
---------------------------------------
1 | anggrek | bunga | 7 | Mati
2 | kamboja | bunga | 5 | Hidup
3 | kol | sayur | 6 | Mati
4 | sawi | sayur | 2 | Mati
5 | apel | buah | 1 | Mati
6 | kacang tanah | kacang | 4 | Hidup
---------------------------------------
```

Gambar 3.8 *Output Daftar Tanaman setelah di Update*

```
=== MENU TANAMAN ===
1. Tambah
2. Lihat
3. Update
4. Hapus
5. Logout
Pilihan: 4
Nomor tanaman: 5
Tanaman berhasil dihapus!
```

Gambar 3.9 *Output Menu Hapus Tanaman*

```
=== MENU TANAMAN ===
1. Tambah
2. Lihat
3. Update
4. Hapus
5. Logout
Pilihan: 5
Logout berhasil.
```

Gambar 3.10 *Output Menu Logout Menu Tanaman*

```
=== LOGIN MENU ===
1. Login
2. Register
3. Keluar
Pilihan: 3

Program selesai.
```

Gambar 3.11 *Output Login Menu Keluar*

E. Langkah - langkah GIT

Untuk menandai file yang ingin disimpan perubahannya.

```
PS C:\Praktikum-apl\post-test\post-test-apl-3> git add .
```

5.1 Git Add

Dapat menyimpan perubahan yang sudah kita add tadi ke dalam riwayat Git.

```
PS C:\Praktikum-apl\post-test\post-test-apl-3> git commit -m "chekup1"
```

5.2 Git Commit

Mengirim commit dari repository lokal ke repository online (GitHub).

```
PS C:\Praktikum-apl\post-test\post-test-apl-3> git push -u origin main --force
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 50.01 KiB | 10.00 MiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/kikaniaagarista-dot/Pratikum-apl.git
 + a1cd0da...343ba70 main -> main (forced update)
branch 'main' set up to track 'origin/main'.
```

5.3 Git Push